LLM Gateway in Go

Portfolio project PRD · Revised October 23, 2026

Purpose and positioning

Build a small OpenAI-compatible HTTP gateway that demonstrates Go concurrency, resilient networking, and measured performance. Applications use one gateway key and model alias; the gateway translates requests, applies limits, and routes calls to an upstream provider.

This is a portfolio engineering project. Success means a reviewer can run it, inspect its tradeoffs, reproduce failures, and verify published results. Broad provider coverage and production product completeness are not release goals.

Skills the project must prove

- **Concurrency:** bounded streaming, context propagation, cancellation, duplicate request suppression, and safe configuration replacement.
- **Resilience:** deadlines, classified errors, bounded fallback, circuit breakers, and explicit dependency-failure behavior.
- **Performance:** reproducible load tests, CPU and heap profiles, and one measured optimization with correctness preserved.

Audience and evidence

A technical reviewer is the primary audience. Platform engineers and application developers provide the usage scenario. The repository must expose a short setup path, readable architecture, executable acceptance scenarios, and honest benchmark limitations.

Release boundary

Version 1 supports text chat through OpenAI and Anthropic adapters, priority routing, SSE streaming, gateway keys, Redis token buckets, opt-in exact caching, SQLite usage records, hot routing reload, and graceful shutdown.

Deployment consists of one Go gateway binary plus Redis and a SQLite file. The gateway is a single binary; the complete system is not dependency-free. A mock provider makes the default demo independent of paid APIs.

Scope and API contract

Supported compatibility subset

Expose POST /v1/chat/completions. Accept model, messages, stream, temperature, top_p, max_tokens, stop, and stream_options.include_usage. Messages contain text only, with system, user, or assistant roles. Support one completion per request; reject n other than 1.

Define accepted ranges and adapter mappings in a checked-in compatibility matrix. Reject unknown fields and unsupported combinations with HTTP 400 and an OpenAI-shaped error. Never silently discard an unsupported option. Validate the request against every configured target before starting fallback.

Return OpenAI-style completion objects and SSE chunks with stable request ID, model alias, choices, finish reason, and usage when known. Separate gateway metadata from provider payloads. An integration example must exercise the supported subset using a pinned OpenAI SDK.

Endpoints

- **GET /v1/models:** aliases allowed by the calling key.
- **GET /healthz:** process liveness. **GET /readyz:** valid configuration and required stores available; provider outages do not make the process unready.
- **GET /metrics:** Prometheus metrics, bound to a local management listener by default.
- **POST /admin/keys, GET /admin/keys, DELETE /admin/keys/{id}:** create, list metadata, and revoke keys.
- **GET /admin/usage:** aggregate request outcomes and upstream token usage by time, key, alias, and provider. Admin endpoints require a separate token.

Error and metadata contract

Errors use {error: {message, type, code}}. Document codes for invalid_request, invalid_api_key, model_not_allowed, rate_limit_exceeded, upstream_failure, all_providers_unavailable, dependency_unavailable, and deadline_exceeded. Define status mappings in API tests.

Successful responses expose X-Request-ID, X-Gateway-Provider, X-Cache, and X-RateLimit-Remaining-Tokens. Headers reflect state when response headers are committed; final usage is recorded separately.

Explicit exclusions

No tools, images, audio, embeddings, structured output, Responses API, admin UI, billing, cost estimation, prompt management, moderation, or semantic caching. Gemini, Ollama, Postgres, weighted routing, tracing, hedging, and distributed coalescing are deferred.

Architecture and bounded work

Request flow

Authenticate and authorize the alias; validate the common request and target capabilities; reserve tokens; inspect the cache; join a coalesced call or execute priority routing; translate the response; reconcile the reservation; record the outcome.

Redis stores shared token buckets and cached responses. SQLite stores hashed gateway keys, request outcomes, and upstream attempt records. In-process state holds immutable routing snapshots, circuit breakers, and singleflight groups.

Provider boundary

Each adapter translates the supported request subset and normalizes responses, usage, and errors. Keep transport and provider-specific decoding inside adapters. Mock fixtures cover each adapter format; one real-provider smoke test per adapter is optional and explicitly enabled.

A streaming adapter returns an owned stream with Next(ctx) and Close() semantics, or an equivalent documented interface. It must expose terminal errors and final usage. The consumer closes the stream; every producer must stop on cancellation. Do not freeze an interface before these lifetimes are proven.

Resource limits

- Enforce request-body, output-token, cached-entry, concurrent-upstream, and active-stream limits. Publish defaults in the sample config.
- Use a small bounded event buffer. Never accumulate an entire stream or create an unbounded queue for a slow client.
- Share configured HTTP clients and transports. Bound connection pools, response-header waits, stream idle time, and downstream writes.
- Use one overall request deadline across retries and backoff. Admission waiting also consumes that deadline.

Usage persistence

Write usage through a bounded worker queue. On queue exhaustion, keep serving and increment an explicit dropped-record counter. Track write failures; never claim billing-grade durability. Shutdown drains the queue within its deadline. Document possible loss on process crash.

Storage and key access

Use SQLite as the only SQL backend. Query key metadata for authentication in version 1; avoid a second key cache and revocation protocol. Serialize writes where needed, set a bounded busy timeout, and measure database contention during load tests.

Streaming and failure policy

Streaming lifecycle

Parse upstream SSE and translate events into the gateway format; this is not raw byte passthrough across provider protocols. Flush complete events promptly. Preserve ordering and emit the normal terminal marker only after successful completion.

Before response commitment, an eligible failure may trigger fallback. Once response headers or body are committed, never switch providers. On a mid-stream failure, attempt an OpenAI-shaped error payload in an SSE data event and close without a success marker. If writing fails, close immediately. Test how the pinned SDK surfaces both cases.

Propagate client disconnect and request cancellation to upstream I/O. This requests remote cancellation; it cannot guarantee that generation or billing stops immediately. Apply write deadlines so slow clients cannot retain resources indefinitely.

Routing and attempt budget

Use ordered priority targets. Version 1 permits at most two upstream attempts total and at most one attempt per target. Each attempt receives the remaining overall deadline, bounded by its configured attempt timeout. Backoff uses capped exponential delay with jitter and never exceeds remaining time.

Fallback is eligible for transport failures, upstream timeouts, HTTP 429, and HTTP 5xx before commitment. Respect Retry-After only when it fits the remaining budget. Invalid requests and upstream authentication or permission failures do not trigger fallback. Client cancellation ends work immediately.

A timeout can leave remote work running and create duplicate provider spend. Record each attempt, including unknown usage. Never promise exactly-once provider execution.

Circuit breakers

Scope breakers by configured provider credential and upstream model. Only transient upstream failures count. Exclude client cancellation, request validation, and local store failures. Bound half-open probes to one concurrent attempt per breaker.

Make thresholds, observation window, open duration, and recovery criteria explicit in configuration. Tests use a fake clock. Breakers are local to each gateway instance; Redis does not coordinate them.

Shutdown

On SIGTERM, mark unready, reject new work, and drain active requests. When the shutdown deadline expires, cancel remaining streams and close transports. Drain usage records within the same finite shutdown budget, then exit.

Token limits and accounting

What the limit means

Each gateway key has a token bucket with capacity (burst tokens), refill rate (tokens per second), and a mandatory maximum output-token bound. Reservations enforce admission against estimated provider work; they are not an exact spending cap.

Estimate input tokens using a documented per-adapter method. Reserve estimated input plus requested maximum output tokens before execution. Reject a reservation larger than bucket capacity. Underestimation can produce a negative balance after reconciliation; new work waits for refill.

Atomic state transitions

Redis Lua scripts atomically refill and reserve, then reconcile by reservation ID. Reconciliation is idempotent and cannot refund twice. Reservation records and bucket state have documented TTLs. Use Redis server time consistently across instances.

Each upstream attempt requires its own reservation. Reconcile a completed attempt before reserving the next. Fallback may therefore stop because no token budget remains. This makes retry cost visible rather than treating multiple provider calls as one.

Known and unknown usage

- **Known usage:** reconcile reserved tokens against provider-reported input plus output tokens. Record the source as provider-reported.
- **Unknown usage:** retain the full reservation debit. Mark usage unknown for timeouts, interrupted streams, or responses without reliable usage.
- **No upstream attempt:** release the reservation for a cache hit, a coalesced follower, or local rejection after admission.
- **Process crash:** retain unresolved debits; expire reservation metadata without issuing automatic refunds. Normal refill restores capacity. Bound reconciliation lifetime to prevent stale refunds.

Two distinct measurements

Client response usage describes the returned completion. Upstream attempt usage describes work performed, including failed attempts where reported. Cache hits and coalesced followers consume no additional upstream tokens. Never count cached response tokens as new provider consumption.

Redis outage policy

Fail closed for new inference requests when the shared limit cannot be checked; return dependency_unavailable. Do not switch to independent local buckets. Existing streams continue. If reconciliation fails, retain the debit and emit an accounting error. Cache access alone may fail open as a miss when reservation operations remain available.

Caching and configuration

Cache contract

Exact caching and coalescing are opt-in and limited to non-streaming requests in version 1. A route and gateway key must both allow reuse. Opt-in explicitly permits replay of a previous stochastic completion.

Build the cache key from gateway key ID, model alias, route revision, adapter schema version, and all effective supported generation inputs. Canonicalize JSON structure only. Preserve message order, text, whitespace, and meaningful distinctions between omitted and explicit values.

Cache only successful complete responses. Store the serving provider and usage provenance. Enforce TTL, maximum entry size, and an explicit Redis memory/eviction configuration. Do not cache errors or share results across gateway keys. Prompts and completions are sensitive cache data even when body logging is disabled.

Coalescing semantics

Use singleflight inside one process only. Concurrent requests on different gateway instances may still make duplicate upstream calls. Redis provides shared caching, not a distributed execution lock.

Each caller may stop waiting independently. Shared execution uses its own bounded context so the initiating caller cannot cancel other waiters. In version 1 it may finish and populate the cache even if all waiters leave; bound this work by deadline. Document and test that tradeoff.

The execution owner retains the upstream reservation. Followers release their own admission reservations. Record each client outcome separately, but associate provider usage only with the shared execution and its attempts.

Hot routing reload

Reload a declarative routing file on SIGHUP. Validate the complete candidate, provider references, supported capabilities, limits, and timeouts before publishing it. Atomically replace an immutable snapshot. Invalid reloads preserve the old snapshot and emit a visible error.

Each request pins one snapshot through completion, including fallback. Increment route revisions when behavior changes to invalidate affected cache entries. Reload changes routing and policies only; provider credential rotation requires restart in version 1.

Minimal configuration

Configure OpenAI and Anthropic credentials through environment variables. Configure one default-chat alias with ordered targets, overall and attempt timeouts, output bound, and cache policy. Ship a validated example and an invalid example used by reload tests.

Observability and acceptance

Required visibility

Emit structured JSON logs using log/slog with request ID, alias, provider, attempt number, duration, outcome, cache/coalescing result, and config revision. Never log gateway secrets, provider credentials, or prompt/completion bodies in version 1.

Prometheus records request and attempt counts, latency, time to first token, active streams, cache hits, coalescing, limit rejections, breaker states, usage-write failures, and dropped usage records. Use bounded provider, alias, and outcome labels. Keep per-key analysis in SQL rather than unbounded metric labels.

Ship one Grafana dashboard covering traffic, errors, latency, stream concurrency, and fallback. Expose pprof only through an opt-in local management listener. OpenTelemetry is deferred.

Release acceptance scenarios

- **Compatibility:** the pinned SDK completes text chat and SSE against both adapter fixtures. Unsupported inputs fail explicitly.
- **Fallback:** a primary transient failure before commitment selects the secondary within the total deadline and two-attempt cap.
- **Streaming:** a mid-stream drop never invokes a second provider or emits a success marker. A slow reader cannot grow buffers without bound.
- **Cancellation:** the mock observes cancellation within one second of client disconnect under local test conditions. Repeated cycles show no sustained goroutine or connection growth.
- **Limits:** concurrent requests across two gateway instances cannot reserve more than available bucket capacity. Reconciliation is idempotent.
- **Cache:** identical overlapping requests in one process make one upstream call. Key isolation, TTL expiry, config revision, and follower cancellation are tested.
- **Reload and shutdown:** invalid reload preserves the last good config; active requests retain their snapshot. Shutdown meets its configured deadline.
- **Dependency failure:** Redis outage rejects new inference calls predictably. SQLite outage and queue overflow produce visible accounting-loss signals.

Test approach

Run unit and integration tests under go test -race. Use fake clocks for limiter/breaker logic, adapter fixtures for protocol translation, and the mock server for streaming and transport failures. Keep paid-provider tests optional and outside deterministic CI.

Benchmarks and portfolio delivery

Reproducible benchmark contract

Ship a seeded mock server with configurable first-token delay, chunk rate, payload size, transient failures, and mid-stream drops. Publish the commands, commit, Go version, hardware, operating system, dependency versions, and relevant gateway configuration.

Run direct-to-mock and gateway tests with matching request payloads, concurrency, connection reuse, upstream behavior, warmup, duration, and offered load. Report achieved throughput, errors, p50/p95/p99 latency, time to first token, CPU, RSS, heap, and active goroutines. Identify load-generator saturation.

Report cache hits, cache misses, coalesced requests, non-streaming, and concurrent streams separately. Show repeated-run variation. A difference between direct and gateway percentiles is a comparative result, not a per-request overhead distribution.

Start with concurrency levels of 1, 10, 100, and 500 where hardware permits. Establish and record a baseline before selecting numeric performance budgets. Do not invent throughput claims. Publish one CPU/heap profile investigation and a before/after change under the same workload.

Five-minute reviewer demo

- Start gateway, Redis, mock provider, and monitoring through one documented command; no paid API key required.
- Run a normal stream, then fail the primary before streaming and show fallback.
- Disconnect a streaming client and show upstream cancellation and resource cleanup.
- Send duplicate non-streaming requests and show one upstream execution; then exceed a token budget.
- Show the dashboard, benchmark summary, one profiling improvement, and known limitations.

Delivery sequence

1. **Protocol foundation:** text contract, two adapters, mock fixtures, auth, and bounded streaming.
2. **Resilience:** deadline budget, fallback, breakers, Redis reservations, and failure tests.
3. **Evidence:** cache/coalescing, reload, SQLite usage, dashboard, load results, and demo.

Definition of done

The release passes deterministic acceptance scenarios and race checks. A fresh reviewer can run the mock-backed demo from the README. The repository includes a compatibility matrix, architecture diagram, benchmark data, three short design decisions (stream ownership, accounting, and coalescing), and clear limitations. Extra providers or product features do not substitute for missing evidence.